

SwiftyRing

SISTEMA DE TIMBRE ESCOLAR INTELIGENTE

SwiftyRing

Documento de requerimientos · v5.0

Aplicación móvil Flutter + módulo físico ESP32 · Backend propio con Vapor (Swift) + PostgreSQL. Especificación funcional y técnica del sistema.

CLIENTE	DESARROLLADO POR	VERSIÓN
I.E. Santa Sofía — Dosquebradas, Colombia	Swifty Devs	5.0
FECHA	**PLATAFORMA**	**CAMBIOS V5.0**
Julio de 2026	Flutter · Vapor · PostgreSQL · ESP32	El backend pasa a ser servidor de identidad de la aplicación institucional.

CONTENIDO

Índice

01 Visión general

02 Arquitectura del sistema

03 Requerimientos funcionales

- 3.1 Gestión de usuarios y autenticación
- 3.2 Módulo de horarios
- 3.3 Timbre manual
- 3.4 Módulo de detección de sismos
- 3.5 Pantalla OLED en módulo físico
- 3.6 Operación offline
- 3.7 Módulo de profesores y notificaciones de clase
- 3.8 Servidor de identidad para módulos externos

04 Requerimientos no funcionales

05 Stack tecnológico

06 Plan de desarrollo por fases

07 Entregables esperados

Cambios frente a la versión 4.0

Al proyecto de grado se sumó un segundo módulo, **Control Baños SS**, desarrollado por otro equipo de aprendices de la misma institución. Los dos módulos se entregan sobre una misma aplicación móvil y una misma base de datos, pero se desarrollan y despliegan por separado.

LO QUE NO CAMBIÓ

Ninguna funcionalidad del Sistema de Timbre Escolar. Las 17 tablas del modelo de datos, los 43 requerimientos funcionales originales y el módulo físico ESP32 quedan exactamente igual. Los cambios de esta versión son de **frontera**: definen qué puede y qué no puede hacer un módulo ajeno que comparte infraestructura.

QUÉ CAMBIÓ	CÓMO ESTABA (V4.0)	CÓMO QUEDA (V5.0)
Rol del backend	Autenticaba únicamente a su propia app	Además actúa como servidor de identidad de la aplicación institucional
Firma de tokens	HMAC-SHA512, llave única compartida	RS256: llave privada firma, llave pública verifica
Claims del JWT	`sub`, `exp`, `rol`	Se agregan `iss`, `aud`, `iat`, `nombre`, `email`
Base de datos	Base propia y exclusiva	Base compartida; el timbre conserva el schema `public` intacto
Permisos de PostgreSQL	Un solo rol con acceso total	Se agrega `proyectobanos` con solo lectura sobre dos tablas
Estructura del código	`lib/features/…`	`lib/core/` + `lib/modules/`, módulo ajeno eliminable
Compatibilidad iOS	iOS 14+	iOS 15+ — impuesto por Xcode 27, que no admite objetivos por debajo de 15.0
Totales	43 RF · 9 RNF	48 RF · 13 RNF

Las condiciones exactas de la frontera entre los dos módulos están en `Docs/04-Integracion/Contrato_Integracion_Timbre_ControlBanos.md`.

01

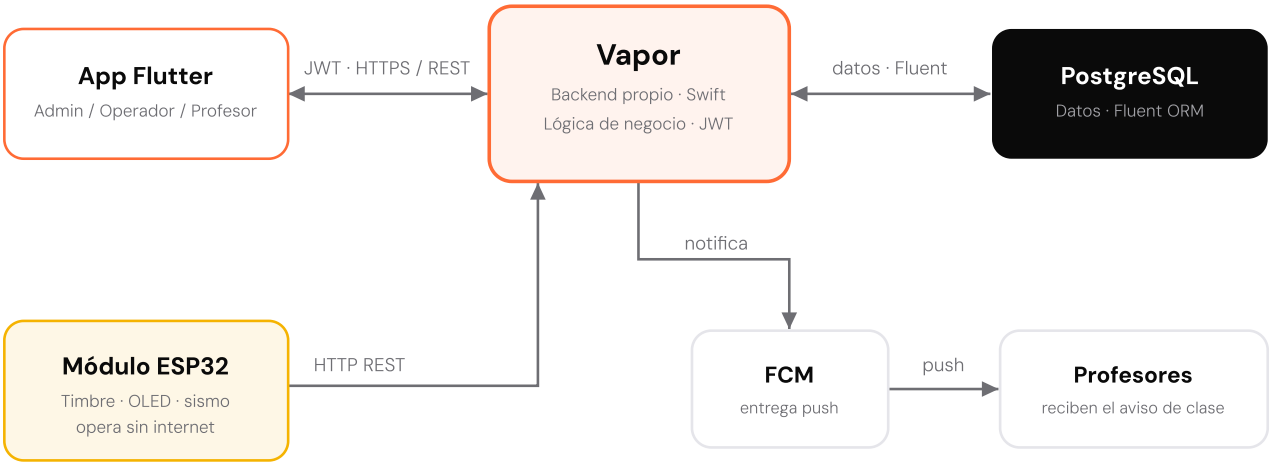
Visión general

Sistema integrado de control de timbre escolar compuesto por una aplicación móvil Flutter y un módulo físico basado en ESP32. El backend es propio: **Vapor + PostgreSQL** alojado en un servidor dedicado, lo que garantiza control total sobre los datos, independencia de plataformas externas y escalabilidad a largo plazo. Las notificaciones push se entregan mediante FCM (Firebase Cloud Messaging) únicamente como canal de entrega, sin depender de ningún otro servicio de Firebase.

02

Arquitectura del sistema

El **backend Vapor** es el centro del sistema: la app y el módulo ESP32 se comunican con él, guarda todo en PostgreSQL y despacha las notificaciones a los profesores por FCM.



El módulo ESP32 llama a Vapor cada vez que el timbre suena —automático, manual o sísmico— y opera de forma autónoma aunque pierda la conexión.

03

Requerimientos funcionales

3.1 · Gestión de usuarios y autenticación

ID	REQUERIMIENTO
RF-01	El sistema contará con autenticación propia mediante JWT (JSON Web Tokens) gestionada desde el backend **Vapor**.
RF-02	Existirá una cuenta de Administrador Principal (Doña Andrea) creada en la inicialización del sistema, no eliminable.
RF-03	El administrador principal podrá crear, editar, suspender y eliminar usuarios secundarios.
RF-04	El sistema manejará tres roles: Administrador, Operador y Profesor.
RF-05	Los operadores podrán consultar horarios y activar el timbre manual, pero no modificar horarios ni gestionar usuarios.
RF-06	El administrador podrá asignar y revocar permisos individuales por usuario.

3.2 · Módulo de horarios

ID	REQUERIMIENTO
RF-07	El sistema permitirá definir un horario habitual semanal con múltiples timbres por día.
RF-08	Cada timbre tendrá: hora, días activos (L–S), etiqueta descriptiva (ej. Inicio de clases) y duración del sonido.
RF-09	El administrador podrá crear, editar y eliminar timbres del horario habitual.
RF-10	El sistema soportará horarios especiales que suspenden temporalmente el horario habitual.
RF-11	Al activar un horario especial, se habilitará un date picker para definir el rango de días de vigencia.
RF-12	Al vencer el rango del horario especial, el sistema retomará automáticamente el horario habitual.
RF-13	Solo el administrador podrá activar, modificar y desactivar horarios especiales.

3.3 · Timbre manual

ID	REQUERIMIENTO
RF-14	La app contará con un botón de timbre manual visible en la pantalla principal.
RF-15	El módulo ESP32 contará con un botón físico de timbre manual.
RF-16	Ambos botones ejecutarán el timbre de forma inmediata, independientemente del horario activo.
RF-17	Cada activación manual quedará registrada en PostgreSQL con usuario, timestamp y origen (app / físico).

3.4 · Módulo de detección de sismos

ID	REQUERIMIENTO
RF-18	El ESP32 contará con un acelerómetro (MPU-6050) para detección de vibraciones sísmicas.
RF-19	El algoritmo de detección deberá distinguir entre sismos y vibraciones de vehículos, usando análisis de frecuencia y duración de la vibración.
RF-20	Un evento sísmico confirmado enviará una notificación push FCM al dispositivo del administrador.
RF-21	La notificación incluirá: hora del evento, intensidad estimada (en g) y duración de la vibración.
RF-22	Al confirmarse un evento sísmico, el módulo ESP32 activará el timbre de forma inmediata y autónoma. Simultáneamente **Vapor** enviará la notificación push al administrador como registro del evento.
RF-23	El historial de eventos sísmicos detectados será consultable desde la app.

3.5 · Pantalla OLED en módulo físico

ID	REQUERIMIENTO
RF-24	El módulo incluirá una pantalla OLED de 1.3" (SH1106).
RF-25	La pantalla mostrará en tiempo real: hora actual, nombre del próximo timbre y tiempo restante.
RF-26	Al activarse el timbre, la pantalla mostrará un indicador visual de timbre activo.
RF-27	Si no hay conexión a internet, la pantalla indicará el modo offline con un ícono visible.

3.6 · Operación offline

ID	REQUERIMIENTO
RF-28	El ESP32 almacenará localmente el horario activo en memoria flash (SPIFFS / LittleFS).
RF-29	La app Flutter utilizará caché local para permitir consulta y edición de horarios sin conexión.
RF-30	Los cambios realizados offline se sincronizarán automáticamente con el backend al recuperar conectividad.
RF-31	El módulo ESP32 operará el timbre automático de forma autónoma aunque pierda conexión al servidor.

3.7 · Módulo de profesores y notificaciones de clase Nuevo en v2.0

ID	REQUERIMIENTO
RF-32	Cada profesor tendrá un perfil en el sistema con: nombre, correo, materia(s) que dicta y token FCM de su dispositivo.
RF-33	El administrador podrá crear, editar y eliminar perfiles de profesores desde la app.
RF-34	Cada profesor tendrá un horario semanal asignado con entradas que incluyen: día, hora de inicio, hora de fin, materia y salón.
RF-35	El administrador podrá cargar y modificar el horario semanal de cada profesor desde la app.
RF-36	Al activarse el timbre (automático, manual o sísmico), **Vapor** consultará qué clase corresponde a continuación para cada profesor según su horario en PostgreSQL.
RF-37	Se enviará una notificación push FCM personalizada a cada profesor con: hora de la próxima clase, materia y salón al que debe dirigirse.
RF-38	Si un profesor no tiene clase en el siguiente período, no recibirá notificación.
RF-39	Los profesores tendrán acceso a la app con rol Profesor, diferente al Administrador y al Operador.
RF-40	La app en modo Profesor mostrará el horario semanal completo del profesor autenticado.
RF-41	La clase en curso se resaltará visualmente con un indicador de color y etiqueta En curso.
RF-42	La próxima clase se resaltará con un indicador secundario y etiqueta Próxima.
RF-43	El horario del profesor se actualizará en tiempo real si el administrador realiza cambios durante el día.

3.8 · Servidor de identidad para módulos externos Nuevo en v5.0

Estos requerimientos no describen funcionalidad que el docente vea en pantalla. Describen lo que el backend debe ofrecer para que otro módulo de la aplicación institucional autentique usuarios **sin acceder nunca a las credenciales**, de modo que el docente use un solo usuario y una sola contraseña para toda la aplicación.

ID	REQUERIMIENTO
RF-44	El backend firmará los tokens JWT con el algoritmo **RS256**, de modo que la llave de verificación pueda entregarse a terceros sin permitirles emitir tokens.
RF-45	El token incluirá los claims `sub`, `exp`, `iat`, `rol`, `nombre`, `email`, `iss` (emisor) y `aud` (audiencias autorizadas).
RF-46	El backend expondrá la llave pública de verificación en el endpoint público `/.well-known/jwks.json`.
RF-47	El backend expondrá un endpoint de solo lectura `GET /api/profesores`, accesible con rol Administrador, con la lista de docentes activos.
RF-48	Ningún módulo externo podrá crear, modificar ni eliminar registros del Sistema de Timbre Escolar a través de la API. La superficie expuesta es exclusivamente de lectura.

POR QUÉ RS256 Y NO HMAC

Con HMAC la llave que firma y la que verifica son la misma: entregarla para que otro módulo valide tokens le permitiría también **fabricarlos**, incluidos tokens con rol Administrador. Con RS256 la llave privada se queda en el servidor del timbre y solo se publica la pública. El módulo externo puede comprobar que un token es auténtico, pero no puede crear uno.

Requerimientos no funcionales

ID	REQUERIMIENTO
RNF-01	La app usará los colores institucionales del colegio como paleta principal.
RNF-02	La interfaz seguirá los lineamientos de Material Design 3.
RNF-03	La app debe ser compatible con Android 8.0+ e iOS 15+.
RNF-04	El tiempo de respuesta entre comando de la app y activación del timbre físico no debe superar 2 segundos con conexión estable.
RNF-05	El envío de notificaciones FCM a todos los profesores debe completarse en menos de 5 segundos desde la activación del timbre.
RNF-06	Toda comunicación entre la app y el backend usará HTTPS con certificado SSL válido.
RNF-07	Las contraseñas se almacenarán en PostgreSQL con hash bcrypt. Nunca en texto plano.
RNF-08	La autenticación se realizará mediante JWT con tiempo de expiración configurable.
RNF-09	El código Flutter seguirá arquitectura Clean Architecture con separación por capas (data / domain / presentation).
RNF-10	**Autonomía.** El Sistema de Timbre Escolar no dependerá en ningún punto de otro módulo de la aplicación institucional: debe compilar, migrar y operar aunque los demás módulos se eliminen por completo. Nuevo
RNF-11	**Aislamiento de datos.** Ningún rol de base de datos ajeno al Sistema de Timbre Escolar tendrá permisos de escritura sobre el schema `public`. La restricción se aplicará con `GRANT/REVOKE` en PostgreSQL, no por convención entre equipos. Nuevo
RNF-12	**Modularidad del código.** La app se organizará en `lib/core/` (compartido) y `lib/modules/<módulo>/`. Ningún archivo de `core/` ni de `modules/timbre_escolar/` podrá importar código de otro módulo. Nuevo
RNF-13	**Custodia de la llave privada.** La llave privada de firma RS256 residirá exclusivamente en el servidor del Sistema de Timbre Escolar, fuera del control de versiones. Solo se distribuye la llave pública. Nuevo

Prueba de aislamiento

RNF-10 y RNF-12 no son declaraciones de intención: se verifican con una prueba concreta antes de cada entrega, sobre una rama descartable.

OPERACIÓN	RESULTADO ESPERADO
`DROP SCHEMA banos CASCADE`	Ninguna tabla del schema `public` se ve afectada
`rm -rf lib/modules/proyecto_banos/`	`flutter analyze` y `flutter build` siguen pasando
`INSERT` sobre `public.usuarios` con el rol ajeno	Falla con *permission denied*

05

Stack tecnológico

App móvil

Framework	`Flutter (Dart)`
State management	`Riverpod / Bloc`
Caché local	`Hive / Isar`
Autenticación	`JWT almacenado en secure storage`
Notificaciones	`flutter_local_notifications + FCM`

Backend

Framework	`Vapor (Swift)`
Base de datos	`PostgreSQL`
ORM	`Fluent (fluent-postgres-driver)`
Autenticación	`JWT con vapor/jwt + Bcrypt`
Servidor	`AlmaLinux — Nginx reverse proxy`
Dominio	`host.swiftydevs.dev`

Notificaciones push

Servicio	FCM — Firebase Cloud Messaging (solo canal de entrega)
Integración	Vapor llama directamente a la API HTTP de FCM
Dependencia Firebase	Ninguna — solo se usa la Server Key de FCM

Módulo físico

Microcontrolador	ESP32 DevKit v1 / v4
Sensor sísmico	MPU-6050 (acelerómetro + giroscopio)
Pantalla	OLED 1.3" I2C (SH1106)
Actuador	Relé de estado sólido 3.3V
Comunicación	HTTP REST hacia Vapor vía WiFi

Plan de desarrollo por fases

FASE	ALCANCE
Fase 1 — Base del proyecto Mes 1	Proyecto Flutter + Vapor configurados · PostgreSQL con modelo de datos · JWT Auth funcionando · Los 3 roles operativos.
Fase 2 — Módulo de timbre Mes 2	Sincronización ESP32 con Vapor · Timbre automático y manual · Operación offline · Pantalla OLED funcionando.
Fase 3 – 4 — Módulo de profesores Mes 3 - 4	CRUD de perfiles y horarios de profesores · Vista semanal con clase en curso · Notificaciones FCM desde Vapor.
Fase 5 — Módulos avanzados Mes 5	Horarios especiales con date picker · Módulo sísmico con algoritmo anti-vehicular · Historial de eventos.
Fase 6 — Pulido y entrega Mes 6	UI con colores institucionales · Pruebas reales con ESP32 físico · Corrección de bugs · Documentación final.

07

Entregables esperados

01 · App Flutter (Android + iOS)

Módulos de usuarios, horarios, timbre manual y vista de profesores.

02 · Backend Vapor

API REST documentada con todos los endpoints del sistema.

03 · Base de datos PostgreSQL

Modelo relacional completo con migraciones y datos iniciales.

04 · Firmware ESP32

Código fuente documentado para el módulo físico.

05 · Manual de usuario

Guía básica para el administrador (Doña Andrea).

06 · Diagrama de flujo

Algoritmo anti-vibración vehicular para el módulo sísmico.

Sistema de Timbre Escolar Inteligente.

Swifty.Devs

DOCUMENTO DE REQUERIMIENTOS V5.0 · DOSQUEBRADAS, COLOMBIA · 2026

BUILT FOR APPLE. DAY 0.